Digital Signal Processing
Systems
Script: Chapter 2 (Systems)

Exercise Sheet 2
with solutions

Learning goals

- Use the operator view $y = S\{x\}$ and decide for a given system whether it is static or dynamic (memory), causal, linear and time-invariant.
- Understand how a two-point lambda controller with integrator works, why it produces a limit cycle, and predict its period from the loop dead time.
- *Bonus (optional):* design numerical test harnesses that *reveal* a property and know their pitfalls (impulse-only tests, small amplitudes, boundary effects); model engine subsystems as dynamic systems: inertia (first-order lag), wall film, transport delay, switching sensor.

Definitions used on this sheet

For a system $y = S\{x\}$ (continuous or discrete time):

- *memoryless (static)*: $y(t_0)$ depends only on $x(t_0)$;
- *causal*: $y(t_0)$ depends only on $x(t)$, $t \le t_0$; equivalently, if $x_1(t) = x_2(t)$ for $t < t_0$, then $y_1(t) = y_2(t)$ for $t < t_0$;
- *linear*: $S\{ax_1 + bx_2\} = aS\{x_1\} + bS\{x_2\}$ for all inputs and constants (superposition = additivity + homogeneity);
- *time-invariant*: $y(t) = S\{x(t)\} \Rightarrow y(t - t_0) = S\{x(t - t_0)\}$ for every shift t_0 (discrete time: integer shifts n_0).

Linearity and time invariance are previewed here; Chapter 3 shows why their combination (LTI) is so powerful.

Exercise 2.1 Properties by hand

PAPER

a) Decide for each system whether it is memoryless, causal, linear and time-invariant (table with yes/no):

S1: $y(t) = 2x(t)$	S2: $y(t) = x^2(t)$	S3: $y[n] = x[n+1] - x[n]$
S4: $y[n] = \frac{1}{3}(x[n] + x[n-1] + x[n-2])$	S5: $y(t) = x(t - T_d)$, $T_d > 0$	S6: $y(t) = \lvert x(t)\rvert$
S7: $y(t) = \int_{-\infty}^{t} x(\tau)\,\mathrm{d}\tau$	S8: $y[n] = n\,x[n]$	S9: $y[n] = x[n] + 0.5$
S10: $y[n] = x[2n]$	S11: $n_k = 1/T_k$ (Sheet 1)	S12: $U = 0.5 + 0.4\tanh\frac{1-\lambda}{0.005}$

b) Prove your answers formally for S6 (linearity), S8 (time invariance) and S10 (causality and time invariance). For S9, show that the system is not linear, but that the *difference* of two outputs depends linearly on the difference of the inputs (*incrementally linear*).

c) An engineer computes the mean speed of one crank revolution as the arithmetic mean of the 58 per-tooth speeds $1/T_k$, treating the gap period like a normal tooth period. Compute the relative error at constant speed. After correcting the gap ($3/T_{\text{gap}}$), is the arithmetic mean of the per-tooth speeds equal to $60/T_{\text{rev}}$? Which property of S11 is responsible, and what is the error for a speed fluctuation of $\pm 1\,\%$ (sinusoidal)?

Solution

a) Property table. Treat every system with the same four questions, always in this order:

Step 1: *Memory.* Which input values does $y(t_0)$ (or $y[n_0]$) need? If it needs only $x(t_0)$, the system is memoryless.

Step 2: *Causality.* If it needs other values: are any of them in the *future*? Then the system is non-causal.

Step 3: *Linearity.* Quick check first: a linear system maps $x = 0$ to $y = 0$. If this holds, test homogeneity ($S\{ax\} = aS\{x\}$) and additivity ($S\{x_1 + x_2\} = S\{x_1\} + S\{x_2\}$). Any nonlinear operation *on the values* of x (square, absolute value, tanh, $1/x$, constant offset) destroys linearity.

Step 4: *Time invariance.* Feed the shifted input $x(t - t_0)$ and compare with the shifted output $y(t - t_0)$. Warning signs for time variance: the time variable appears *outside* the argument of x (like the factor n in S8), or the time axis itself is transformed (like $2n$ in S10).

This gives (M = memoryless, C = causal, L = linear, TI = time-invariant):

system	M	C	L	TI	remark
S1 $2x$	yes	yes	yes	yes	amplifier
S2 x^2	yes	yes	no	yes	$S\{2x\} = 4S\{x\}$
S3 $x[n+1] - x[n]$	no	no	yes	yes	forward difference uses the future
S4 3-point MA	no	yes	yes	yes	memory of 2 samples
S5 delay	no	yes	yes	yes	transport delay (lambda!)
S6 $\lvert x\rvert$	yes	yes	no	yes	rectifier
S7 integrator	no	yes	yes*	yes	*initially at rest
S8 $n\,x[n]$	yes	yes	yes	no	gain changes with time
S9 $x + 0.5$	yes	yes	no	yes	affine, $S\{0\} \neq 0$
S10 $x[2n]$	no	no	yes	no	down-sampler
S11 $1/T$	yes	yes	no	yes	speed from tooth period
S12 tanh	yes	yes	no	yes	switching lambda sensor

Comments on the less obvious entries:

- *S3, S4, S5* have memory: they need x at times other than n. Only S3 needs a *future* value ($x[n+1]$) and is therefore non-causal.
- *S7*: $y(t) = \int_{-\infty}^{t} x\,\mathrm{d}\tau$ needs the whole past. It is linear only if the integrator starts at rest. With an initial value $y_0 \neq 0$ it behaves like S9 (an offset that does not scale with x).
- *S11*: each speed value is computed from one period T_k only, so it is memoryless *in the tooth index* k, but $1/T$ is nonlinear.
- *S12*: the sensor characteristic is static (the sensor lag is a separate block), but extremely nonlinear: it saturates at 0.1 and 0.9 V.

b) Proofs. To show that a property *fails*, one concrete counterexample is enough. To show that it *holds*, you need a proof for all inputs and all constants.
S6, linearity.

Step 1: Homogeneity requires $S\{ax\} = aS\{x\}$ for every a.

Step 2: Choose $x_1(t) = 1$ and $a = -1$: $S\{-x_1\} = |-1| = 1$, but $-S\{x_1\} = -1$. Homogeneity fails ⇒ S6 is not linear.

Step 3: Additivity fails as well: $x_1 = 1$, $x_2 = -1$ gives $S\{x_1 + x_2\} = 0 \neq S\{x_1\} + S\{x_2\} = 2$.

Step 4: For comparison, TI holds: $x_2(t) = x(t - t_0)$ gives $y_2(t) = |x(t - t_0)| = y(t - t_0)$.

How a time-invariance proof works. The shifted input $x_2[n] = x[n - n_0]$ is not derived – you *choose* it, because the test requires it. Time invariance means: the system behaves today as it did yesterday; the same input applied n_0 samples later gives the same output, n_0 samples later. So compare two experiments:

(i) original: input $x[n]$, output $y[n] = S\{x\}[n]$;

(ii) delayed: the same signal fed in n_0 samples later. What x had at time m, x_2 must have at time $m + n_0$. With $n = m + n_0$, i.e. $m = n - n_0$: $x_2[n] = x[n - n_0]$ (the minus sign means a delay, a shift to the right).

The system is time-invariant if the response to the delayed input equals the delayed original response, $S\{x_2\}[n] = y[n - n_0]$, for every x and every n_0.
S8, time invariance.

Step 1: Shifted input: $x_2[n] = x[n - n_0]$.

Step 2: Response to it: $y_2[n] = n\,x_2[n] = n\,x[n - n_0]$.

Step 3: Shifted output: $y[n - n_0] = (n - n_0)\,x[n - n_0]$.

Step 4: The two differ by $n_0\,x[n - n_0] \neq 0$ for $n_0 \neq 0 \Rightarrow$ time-variant. The gain n depends on the absolute time. The signal x is shifted, the factor n is not.

Step 5: *Example.* Impulse at $n = 2$: $x[2] = 1$, so $y[2] = 2 \cdot 1 = 2$. Feed it $n_0 = 3$ samples later: $x_2[5] = 1$, so $y_2[5] = 5 \cdot 1 = 5$. The delayed original response is $y[5 - 3] = y[2] = 2 \neq 5$: the same impulse is amplified more strongly when it arrives later.

Step 6: S8 is nevertheless linear: $n\,(ax_1[n] + bx_2[n]) = a\,nx_1[n] + b\,nx_2[n]$.

S10, causality and time invariance.

Step 1: Causality: $y[1] = x[2]$, $y[5] = x[10]$, ...: for $n > 0$ the output needs a *future* input $\Rightarrow$ non-causal. For the same reason the output depends on $x[2n] \neq x[n] \Rightarrow$ memory.

Step 2: Shift by one: $x_2[n] = x[n - 1]$ gives $y_2[n] = x_2[2n] = x[2n - 1]$.

Step 3: Shifted output: $y[n - 1] = x[2(n - 1)] = x[2n - 2]$.

Step 4: $x[2n - 1] \neq x[2n - 2]$ in general $\Rightarrow$ time-variant.

Step 5: Even shifts $n_0 = 2m$ work: $y_2[n] = x[2n - 2m] = y[n - m]$, i.e. the output is shifted by $n_0/2$. Such a system is called *periodically time-variant.*

S9, incremental linearity.

Step 1: Quick check: $S\{0\} = 0.5 \neq 0 \Rightarrow$ not linear. (Homogeneity: $S\{ax\} = ax + 0.5 \neq a(x + 0.5)$.)

Step 2: Take two inputs: $y_1 - y_2 = (x_1 + 0.5) - (x_2 + 0.5) = x_1 - x_2$.

Step 3: The *difference* of the outputs is a linear function (here the identity) of the difference of the inputs: S9 is incrementally linear.

This is how all "linearised" engine models work: small deviations from an operating point are transferred linearly, although the system itself is affine or nonlinear.

c) Mean speed from per-tooth speeds.

Step 1: *Per-tooth speed.* A normal tooth spans $6° = \frac{1}{60}$ revolution. With the period T_k in seconds: $n_k = \frac{1/60 \text{ rev}}{T_k} \cdot 60\,\text{s/min} = \frac{1}{T_k}$ in rpm.

Step 2: *Gap.* The gap spans 18°, i.e. three teeth: $T_{\text{gap}} = 3T$ at constant speed $n = 1/T$. Treated like a normal tooth, it contributes $1/T_{\text{gap}} = n/3$.

Step 3: *Mean over the 58 periods* (57 normal teeth + gap):

$$\bar{n} = \frac{57\,n + n/3}{58} = 0.98851\,n, \qquad \text{error } -1.15\,\%.$$

Step 4: *Gap corrected* ($3/T_{\text{gap}}$): all 58 values equal n, so at constant speed the mean is exact.

Step 5: *Speed fluctuation.* The physically correct mean speed is "angle travelled / time" $= 60/T_{\text{rev}}$. Model the fluctuation as $n(\theta) = n_0(1 + \varepsilon \sin 2\theta)$, $\varepsilon = 0.01$. The per-tooth speeds sample $n(\theta)$ at equally spaced *angles*; the sine averages to zero over a revolution, so their mean is n_0.

Step 6: *Revolution time.* The time for one revolution is the integral of $\mathrm{d}\theta/\omega$, i.e. the mean of $1/n(\theta)$ over the angle:

$$T_{\text{rev}} = \frac{60}{n_0}\,\overline{\frac{1}{1+\varepsilon\sin 2\theta}} \approx \frac{60}{n_0}\,\overline{1-\varepsilon\sin 2\theta+\varepsilon^2\sin^2 2\theta} = \frac{60}{n_0}\Big(1+\frac{\varepsilon^2}{2}\Big),$$

using $\overline{\sin} = 0$ and $\overline{\sin^2} = \frac{1}{2}$. Hence $60/T_{\text{rev}} \approx n_0(1 - \varepsilon^2/2)$.

Step 7: *Numbers.* The arithmetic mean of the per-tooth speeds is too high by $\varepsilon^2/2 = 5 \cdot 10^{-5}$, i.e. 0.15 rpm at 3000 rpm – exactly the offset seen on Sheet 1.

Step 8: *Responsible property.* S11 ($1/T$) is *nonlinear* and convex. For a convex function the mean of the function values is larger than the function of the mean (Jensen's inequality): $\overline{1/T} \geq 1/\overline{T}$. Averaging and the nonlinearity do not commute.

Correct procedure: average in the period domain (sum of the T_k) and invert *once*: $60/T_{\text{rev}}$.

Exercise 2.2 Closed lambda loop

WOKWI

Project `code/wokwi/sheet02_lambda/`: Arduino Uno and the custom chip `lambda-probe`. Pin 9 sends the fuel command as PWM (`analogWrite`, 490 Hz, duty $= q/2$), A0 reads the sensor voltage `O2`, A1 the ground truth `LAM` $= \lambda/2$. The chip's sliders set the unmetered air (slider − 10 in %, default +5 %) and the transport delay (default 200 ms).

How a two-point controller works. The goal is the stoichiometric mixture $\lambda = 1$. The ECU does not know the fuel quantity q^* that achieves it, because part of the air is not measured (leaks, sensor and injector tolerances, ageing): with unmetered air a, $\lambda = (1 + a)/q$ and $q^* = 1 + a$. The switching sensor (S12 of Exercise 2.1) delivers only *one bit* of information: $U > 0.45$ V means "rich" ($\lambda < 1$, too much fuel), $U < 0.45$ V means "lean" ($\lambda > 1$, too little fuel) – but not *how* rich or lean. The controller therefore uses the simplest possible rule. In every controller period $T_s = 10$ ms:

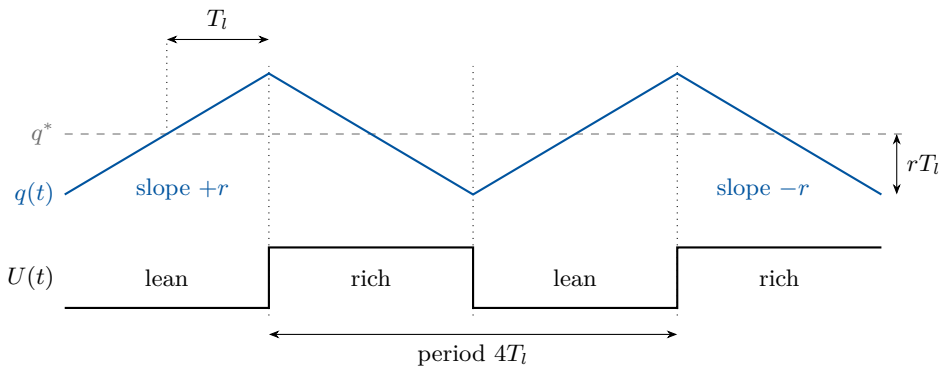
- sensor rich ⇒ decrease q by rT_s (ramp down with slope $-r$),
- sensor lean ⇒ increase q by rT_s (ramp up with slope $+r$),
- optionally, at the moment the sensor *switches*, an additional jump of q by P towards the other side ($-P$ on lean→rich, $+P$ on rich→lean).

"Two-point" means that the controller output (here the slope $\dot{q}$) takes only two values, $\pm r$, like a relay or a thermostat. "With integrator" means that q is the integral of this output: as long as the sensor reports the same state, q keeps moving in the same direction, so it always moves towards q^*, whatever the value of a. On average the controller therefore finds q^* by itself (integral action).

Why it oscillates. The sensor sees a change of q only after the transport delay T_d and the two lags (wall film $\tau_f = 20$ ms, sensor $\tau_s = 30$ ms), i.e. after an effective dead time $T_l \approx T_d + \tau_f + \tau_s$. When q crosses q^*, the sensor still reports the old state for T_l, and q keeps ramping in the wrong direction: it overshoots by rT_l. After the switching, q needs another T_l to return to q^*, and the sensor sees this crossing only T_l later. Each half period therefore lasts $2T_l$:

$$T_{\text{period}} \approx 4\,T_l, \qquad q = q^* \pm rT_l, \qquad \lambda \approx 1 \mp \frac{rT_l}{q^*}.$$

The oscillation neither grows nor decays – a *limit cycle*. The jump P removes part of the overshoot immediately after each switching, so q reaches q^* earlier (by P/r); the period shortens to $4T_l - 2P/r$.



Limit cycle without jump ($P = 0$): U is the sign of $q - q^*$, delayed by T_l.

a) Implement the two-point controller with integrator ($r = 0.2$/s) in `loop()`. Start the simulation and observe the limit cycle in the Serial Monitor (`u_o2`, `lambda`, `q`). Copy 5 s of data into a spreadsheet (or into JSLinux, see the bonus section) and plot λ and U; compare with Fig. 1.3 of the script and with the sketch above. Predict period and λ range for $T_d = 200$ ms and check them.

b) Measure the oscillation period (column `period_ms`) for transport delays of 100, 200, 400 and 800 ms and plot period over T_d. Determine slope and intercept and interpret them.

c) Move the air slider from +5 % to −10 % and to +10 %. Describe the transient and the new mean value of q.

d) Add the jump `PJUMP` = 0.03, then reduce `RATE` to 0.1 with `PJUMP` = 0.025. Compare period and λ amplitude. Why can no two-point controller make the oscillation disappear, and what would a continuous (PI) lambda controller need?

Solution

a) Controller and limit cycle. Translate the rule of the box above line by line into `loop()`:

Step 1: *Read the sensor* and convert it to volts: $u = \text{ADC} \cdot 5/1023$ (line 38 of the template).

Step 2: *Decide rich or lean*: `rich = u > U_THR` with $U_{\text{THR}} = 0.45$ V.

Step 3: *Jump at a switching* (only if `PJUMP` $\neq 0$): if `rich` differs from the value of the last period, move q by P towards the other side. The lean→rich switching is also the moment to measure the period with `millis()`.

Step 4: *Integrator ramp*: change q by $\mp rT_s = \mp 0.2 \cdot 0.01 = \mp 0.002$ per period.

Step 5: *Limit* q to $0.5 \ldots 1.5$, so that the duty cycle stays in $0.25 \ldots 0.75$.

Step 6: *Output*: duty $= q/2$, i.e. `analogWrite` value $255 \cdot q/2 = 127.5\,q$ (rounded).

Reference solution (`code/solutions/wokwi/sheet02_lambda/sketch.ino`):

```
41    /* ---- two-point controller with integrator ---------------------- */
42    bool rich = u > U_THR;
43    if (rich != rich_old) {
44        q += rich ? -PJUMP : PJUMP;                    /* P jump */
45        if (rich) {                                    /* lean -> rich */
46            unsigned long now = millis();
47            if (t_switch) period_ms = now - t_switch;
48            t_switch = now;
49        }
50    }
51    q += (rich ? -RATE : RATE) * (TS_US * 1e-6f);      /* I ramp */
52    if (q < 0.5f) q = 0.5f;
53    if (q > 1.5f) q = 1.5f;
54    rich_old = rich;
```

```
55
56      /* duty = q / 2, 8-bit PWM */
57      analogWrite(INJ_PIN, (int)(127.5f * q + 0.5f));
```

What you should see:

- `u_o2` switches between ≈ 0.1 V and 0.9 V (ADC values ≈20 and 184), almost rectangular as in Fig. 1.3.
- q is a triangle around $q^* = 1.05$. Because one PWM step is $2/255 = 0.0078$ and the ramp adds only 0.002 per period, q changes in small stairs (one step about every 40 ms).
- λ is a triangle as well (ramps of q, slightly rounded by the wall film). The ground truth via A1 has a resolution of only $2 \cdot 4.9\,\text{mV} \approx 0.01$ in λ, so its extreme values scatter by ± 0.01.
- The first period after the start is longer: q starts at 1.0 and must first ramp up to $q^* = 1.05$.

Prediction for $T_d = 200$ ms. Effective dead time $T_l = T_d + \tau_f + \tau_s = 200 + 20 + 30 = 250$ ms. Period $4T_l = 1.0$ s (the offline simulation of Bonus Exercise 2.B2 c) with the same model as the chip gives 0.98 s). Overshoot of q: $rT_l = 0.2 \cdot 0.25 = 0.05$, so $\lambda \approx 1 \mp 0.05/1.05 = 1 \mp 0.048$, i.e. $0.952 \ldots 1.048$. Measured in the model: $0.958 \ldots 1.046$.

b) Period over transport delay.

Step 1: *Measure.* Expected values (the reference sketch run on the PC against a replica of the chip model):

T_d / ms	period	$4(T_d + 50\,\text{ms})$	λ (from A1)
100	0.58 s	0.60 s	0.978…1.026
200	1.00 s	1.00 s	0.958…1.046
400	1.80 s	1.80 s	0.919…1.095
800	3.40 s	3.40 s	0.860…1.193

In Wokwi the values may deviate by one controller period (10 ms) per switching. The offline model of Bonus Exercise 2.B2 gives up to 20 ms less, because there the controller sees the sensor value of the current millisecond (no extra sampling delay).

Step 2: *Fit a straight line* $T_{\text{period}} = m\,T_d + c$ (least squares over the four points): $m = 4.02$, $c = 0.19$ s.

Step 3: *Slope.* $m \approx 4$: per period the sensor switches twice, and each half period contains the dead time twice (the overshoot builds up during T_l, and the return to q^* is seen only T_l later). So every millisecond of transport delay adds 4 ms to the period.

Step 4: *Intercept.* $c/4 \approx 47$ ms is the part of T_l that is not transport delay: the two lags, $\tau_f + \tau_s = 50$ ms (a ramp passing a first-order lag τ is delayed by τ). The small difference comes from the 8-bit PWM stairs of q and the 10 ms controller period.

Step 5: λ *amplitude* grows with T_l as predicted: $\pm rT_l/q^* = \pm 0.029,\ 0.048,\ 0.086,\ 0.162$ for the four delays. At large T_d the range becomes asymmetric ($0.860 \ldots 1.193$), because $\lambda = q^*/q$ is a nonlinear function of q.

c) Change of the unmetered air.

Step 1: *New target.* At −10 % air the stoichiometric quantity drops to $q^* = 0.90$. At the moment of the change $q \approx 1.05$, so the mixture is suddenly rich: $\lambda = 0.90/1.05 = 0.857$.

Step 2: *Dead time.* The sensor reports rich only after the dead time (simulation: 0.21 s after the step).

Step 3: *Ramp.* The integrator now ramps q down with 0.2 s. It needs $(1.05 - 0.90)/0.2 = 0.75$ s to reach q^*.

Step 4: *Overshoot.* The sensor sees the crossing only T_l later, so lean is reported again after about $0.21 + 0.75 + 0.25 \approx 1.2\,\mathrm{s}$ (simulation `air_step()` in the reference solution of Bonus 2.B2: 1.25 s).

Step 5: *New limit cycle* around the mean $q = 0.900$. At +10 % the mixture becomes lean, q ramps up and settles around mean $q = 1.098$. The overshoot of q (rT_l) is the same; the λ amplitude rT_l/q^* changes only slightly with q^*.

The integrator removes any constant disturbance: the mean of q always ends at $q^* = 1 + a$, without knowing a. This is the purpose of the lambda loop (adaptation of fuel metering errors, air leaks, ageing).

d) Jump P.

Step 1: $P = 0.03$, $r = 0.2/s$. After each switching the jump undoes P of the overshoot $rT_l = 0.05$; q^* is reached $P/r = 0.15\,\mathrm{s}$ earlier in each half period. Prediction $4T_l - 2P/r = 1.0 - 0.3 = 0.70\,\mathrm{s}$; model: 0.70 s. The peak deviation is still rT_l (it builds up *before* the switching), so λ stays at $0.958 \ldots 1.045$: faster, but not smaller.

Step 2: $P = 0.025$, $r = 0.1/s$. Halving r halves the overshoot: $rT_l = 0.025$, λ $0.978 \ldots 1.027$ (model). Without jump the period would stay $4T_l = 1.0\,\mathrm{s}$ (it does not depend on r); the jump shortens it: prediction $1.0 - 2 \cdot 0.025/0.1 = 0.5\,\mathrm{s}$, model 0.58 s.

Step 3: *Why the deviation in step 2?* Here $P \approx rT_l$: the jump lands almost exactly on q^*. The rule "a lag delays a ramp by τ" holds for ramps, not for jumps: after a jump the lagged signal approaches its new value only exponentially, so it reaches q^* later than predicted. (It is not the 8-bit PWM: without quantisation the model gives 0.60 s as well.) Result: half the amplitude at a higher frequency than without jump.

Step 4: *Why the oscillation cannot disappear.* The sensor delivers only the *sign* of $\lambda - 1$ (relay characteristic). The controller must keep moving q to find out on which side of q^* it is. Relay + integrator + dead time always yield a limit cycle of period $\approx 4T_l$; r and P only change its amplitude and period.

Step 5: *What a PI controller would need.* A continuous controller needs a *proportional* measure of $\lambda - 1$, i.e. a wide-band (linear) lambda sensor. With the switching sensor, the oxygen storage of the catalytic converter tolerates the small, fast oscillation – one reason why period and amplitude must be kept small, i.e. why T_d matters.

Deliverables

2.1: property table, proofs. 2.2: sketch, plot of λ and U over 5 s, period over T_d (table + plot), results of c) and d). Bonus 2.B1 (optional): test program, property table with numerical evidence, $h[n]$ of the LTI systems. Bonus 2.B2 (optional): program, values of τ, response times, prediction formula and comparison table.

Bonus: JSLinux (optional)

For interested students: the following exercises identify unknown systems by experiment and simulate the lambda path offline in C. Bonus Exercise 2.B2 c) derives the limit cycle of Exercise 2.2 in detail.

Setup

The following exercises run in JSLinux, a Linux system in the browser. If you have not done Bonus Exercise 0.B1, set it up now (about 10 min):

1. Open `https://bellard.org/jslinux`, start the *x86 Alpine Linux* (console) virtual machine

and check the compiler with `gcc -version` (or `tcc -v`).

2. Upload the course headers `engine_signals.h`, `asciiplot.h`, `csvio.h` (`code/common/`) and the files of the exercise (e.g. `code/jslinux/sheet02/`) with the upload button below the terminal; check with `ls` that they arrived. Headers and program must be in the same directory.
3. Compile and run, e.g.

```
gcc -O2 -o blackbox blackbox.c -lm
./blackbox
```

4. The VM forgets all files when the page is reloaded: keep your sources on your own computer and upload them again. Text from Wokwi (Serial Monitor) gets into the VM with `cat > data.csv`, paste, `Ctrl-D`; `export_file data.csv` downloads a file from the VM to your computer.

Exercise 2.B1 Black-box system identification

JSLINUX

The header `code/jslinux/sheet02/mystery.h` contains five unknown systems

`void Sk(const double *x, double *y, int N);` $k = 1, \ldots, 5$

(input $x[0..N-1]$, $x[n] = 0$ outside, output $y[0..N-1]$). One of them is LTI; the others violate exactly one or two of the properties of Exercise 2.1. **Do not read the header** – find the properties by experiments, as you would with a supplier's ECU function delivered as object code. The program `code/jslinux/sheet02/blackbox.c` plots the responses to unit impulses at $n_0 = 30$ and $n_0 = 31$ and contains a (weak) homogeneity test with scaled impulses as an example.

a) Superposition test: compare $S\{ax_1+bx_2\}$ with $aS\{x_1\}+bS\{x_2\}$ for random signals and random weights, at amplitude 1 and 0.01. Why is the impulse test of the template not sufficient, and why two amplitudes?

b) Shift test: compare $S\{x[n-k]\}$ with $y[n-k]$ for $k = 1 \ldots 4$. Why are the first and last samples of the test signal set to zero?

c) Causality test: two inputs that agree for $n < n_0$ and differ afterwards.

d) Memory test: two inputs that differ only at n_0. Why must the test be repeated for several n_0?

e) Report a property table with the numerical evidence (largest deviation of each test). For the LTI systems determine $h[n]$ from the plots and give the difference equation. Why does an "impulse response" not characterise the other systems?

Solution

How to build a property test. All four tests follow the same recipe:

Step 1: Take the definition of the property (Exercise 2.1) and read it as an experiment: "if the inputs are related like this, the outputs must be related like that".

Step 2: Generate inputs that satisfy the *if* part – with random signals, not with special ones.

Step 3: Run the system on all inputs and compute the largest deviation from the *then* part.

Step 4: Repeat with many random inputs (`NTRY` = 20) and keep the largest deviation. Deviations below a tolerance (10^{-9}) are rounding errors.

Example, the superposition test of the reference solution (`code/solutions/jslinux/sheet02/blackbox.c`):

```
/* ---- (a) superposition: S{a x1 + b x2} = a S{x1} + b S{x2} ? ---------
   returns the largest deviation relative to the output amplitude       */
static double test_linear(system_t S, double amp)
{
```

```
56      double x1[N], x2[N], x3[N], y1[N], y2[N], y3[N], d[N], e = 0.0, r;
57      int t, n;
58      for (t = 0; t < NTRY; t++) {
59          double a = 2.0 * urand(), b = 2.0 * urand();
60          rand_signal(x1, amp, 0);
61          rand_signal(x2, amp, 0);
62          for (n = 0; n < N; n++) x3[n] = a * x1[n] + b * x2[n];
63          S(x1, y1, N); S(x2, y2, N); S(x3, y3, N);
64          for (n = 0; n < N; n++) d[n] = y3[n] - (a * y1[n] + b * y2[n]);
65          r = maxabs(d, N) / (maxabs(y3, N) + 1e-300);
66          if (r > e) e = r;
67      }
68      return e;
69  }
```

Line 62 builds $x_3 = ax_1 + bx_2$, line 64 the deviation $S\{x_3\} - (aS\{x_1\} + bS\{x_2\})$, line 65 divides by the output amplitude so that the result is comparable for $A = 1$ and $A = 0.01$. The shift test:

```
82  /* ---- (b) time invariance: S{x[n-k]} = y[n-k] ? ------------------- */
83  static double test_shift(system_t S)
84  {
85      double x[N], xs[N], y[N], ys[N], e = 0.0, d;
86      int t, k, n;
87      for (t = 0; t < NTRY; t++)
88          for (k = 1; k <= 4; k++) {
89              rand_signal(x, 1.0, 8);
90              for (n = 0; n < N; n++) xs[n] = n >= k ? x[n - k] : 0.0;
91              S(x, y, N); S(xs, ys, N);
92              for (n = k; n < N; n++) {
93                  d = fabs(ys[n] - y[n - k]);
94                  if (d > e) e = d;
95              }
96          }
97      return e;
98  }
```

Output of the program:

```
 sys  linear(A=1) linear(A=0.01) imp-scale   shift      causal     memory
 S1    0.00e+00   0.00e+00       0.00e+00   1.96e+00   0.00e+00   1.00e+00
 S2    3.78e-16   3.49e-16       0.00e+00   0.00e+00   0.00e+00   2.50e-01
 S3    1.55e+00   3.58e-04       1.29e+00   0.00e+00   0.00e+00   0.00e+00
 S4    0.00e+00   2.47e-16       0.00e+00   0.00e+00   4.61e-01   2.50e-01
 S5    8.43e-01   8.48e-03       0.00e+00   0.00e+00   0.00e+00   6.21e-01

 sys  linear  time-inv  causal  memoryless
 S1   yes     NO       yes     NO
 S2   yes     yes      yes     NO
 S3   NO      yes      yes     yes
 S4   yes     yes      NO      NO
 S5   NO      yes      yes     NO
```

For checking, the systems are (from `code/solutions/jslinux/sheet02/mystery_gen.c`):

	system	properties
S1	$y[n] = x[n - (n \bmod 3)]$ (sample & hold every 3 samples)	linear, time-variant, causal, memory
S2	$y[n] = 0.75\,y[n-1] + 0.25\,x[n-1]$	LTI, causal, memory (IIR)
S3	$y[n] = \tanh(x[n])$	nonlinear, memoryless, TI, causal
S4	$y[n] = 0.25x[n-1] + 0.5x[n] + 0.25x[n+1]$	LTI, *non-causal*, memory
S5	$y[n] = 0.625x[n] + 0.375x[n-1] + 0.25\,x[n]x[n-1]$	nonlinear, TI, causal, memory

a) Superposition.

Step 1: *Impulse test of the template* ($S\{3\delta\}$ vs. $3S\{\delta\}$, column `imp-scale`): detects only S3.

Step 2: *Why S5 passes it.* For a single impulse $x[n]x[n-1] = 0$ at every n: an impulse never excites products of *different* samples. The nonlinear term of S5 is invisible.

Step 3: *Random signals with random weights* excite all such terms: S5 now shows a deviation of 0.84.

Step 4: *Why two amplitudes.* S3 ($\tanh x \approx x - x^3/3$) is almost linear for small inputs: relative deviation $3.6 \cdot 10^{-4}$ at $A = 0.01$ vs. 1.55 at $A = 1$. Likewise S5: $8.5 \cdot 10^{-3}$ vs. 0.84 (the quadratic term is $\propto A^2$, the linear part $\propto A$, so their ratio is $\propto A$).

Step 5: *Conclusion.* Every nonlinear system looks linear for small enough signals (small-signal linearisation). A test at small amplitude alone with a coarse tolerance could miss the nonlinearity; the large amplitude reveals it.

b) Shift test.

Step 1: Build $x_s[n] = x[n-k]$ (line 90) and compare $S\{x_s\}[n]$ with $y[n-k]$ for $n \geq k$.

Step 2: *Why zero guard samples.* The arrays have finite length N, with $x[n] = 0$ outside. Shifting a signal that is non-zero up to the end pushes samples out of the array, and zeros are pushed in at the start. A non-causal LTI system such as S4 (reads $x[n+1]$) would see this boundary difference and be wrongly classified as time-variant. With 8 zeros at both ends (line 89), shifts up to 4 lose nothing.

Step 3: *Result.* Only S1 fails (deviation 1.96): shifting the input by one sample changes *which* samples are held.

c) Causality.

Step 1: Two inputs that are equal for $n < n_0$ and different (random) from n_0 on.

Step 2: A causal system must give equal outputs for $n < n_0$.

Step 3: Only S4 reacts before the inputs differ: deviation 0.46 at $n = n_0 - 1$, caused by the weight 0.25 of $x[n+1]$.

d) Memory.

Step 1: Two inputs that differ only at n_0 (by 1).

Step 2: A memoryless system may change its output only at n_0.

Step 3: *Why several n_0.* For S1 a change at n_0 with $n_0 \bmod 3 \neq 0$ has *no* effect at all (the sample is never read); with $n_0 \bmod 3 = 0$ it affects $n_0 \ldots n_0 + 2$. A single unlucky n_0 would report "memoryless". The program therefore uses $n_0 = 20, 21, 22$.

Step 4: *General lesson.* A test can only falsify a property. A passed test is evidence, not proof – use many random trials.

e) Impulse responses of the LTI systems.

Step 1: *S2.* The plot for $n_0 = 30$ shows 0 at $n = 30$ and then 0.25, 0.1875, 0.141, ... from $n = 31$ on. Each value is 0.75× the previous one: $h[n] = 0.25 \cdot 0.75^{n-1}$ for $n \geq 1$, $h[n] = 0$ for $n \leq 0$.

Step 2: A geometric sequence comes from a recursion $y[n] = 0.75\,y[n-1] + \ldots$; the first value 0.25 one sample late gives the input term $0.25\,x[n-1]$: $y[n] = 0.75y[n-1] + 0.25x[n-1]$, an exponential smoother with one sample delay.

Step 3: *S4.* The plot for $n_0 = 30$ shows 0.25, 0.5, 0.25 at $n = 29, 30, 31$: $h[-1] = 0.25$, $h[0] = 0.5$, $h[1] = 0.25$, i.e. $y[n] = 0.25x[n+1] + 0.5x[n] + 0.25x[n-1]$, a centred, non-causal smoother.

Step 4: *The other systems.* Their response to an impulse depends on *where* the impulse is (S1: three ones for $n_0 = 30$, nothing for $n_0 = 31$) or on *how large* it is and which other samples are present (S3, S5). The formula $y = h * x$ (Chapter 3) requires linearity *and* time invariance; otherwise one impulse response does not describe the system.

Exercise 2.B2 The engine as a dynamic system

JSLINUX

Program: `code/jslinux/sheet02/engine_dyn.c`.

a) *Crankshaft inertia.* With fuel quantity q (1 = nominal), the engine speed obeys $\tau\,\dot{n} = Kq - n$, $\tau = 0.4\,\text{s}$, $K = 2000\,\text{rpm}$. Simulate a fuel step $1 \to 1.5$ at $t = 0.5\,\text{s}$ with the Euler method ($T = 1\,\text{ms}$) and with the exact discretisation $n[k] = a\,n[k-1] + (1-a)Kq[k-1]$, $a = \mathrm{e}^{-T/\tau}$ (derive it!). Determine τ from the 63 % point and compare both methods with the analytic solution at $t = 0.6\,\text{s}$. Is the engine (fuel → speed) static or dynamic?

b) *Lambda path.* Implement `lp_step()` (one step = 1 ms) – it is the model inside the Wokwi chip `lambda-probe`: $q \to$ wall film (first-order lag, 20 ms) $\to \lambda = (1 + a_{\text{air}})/q_f \to$ transport delay T_d $\to$ sensor lag (30 ms) $\to U = 0.5 + 0.4\tanh((1-\lambda)/0.005)$. Classify each block (static/dynamic, linear, TI). Simulate fuel steps $1.00 \to 1.10$ and $1.00 \to 1.20$ ($a_{\text{air}} = +5\,\%$, $T_d = 200\,\text{ms}$) and measure the time until U crosses 0.45 V. Which observations prove that the chain is nonlinear? Under which operating condition is it not time-invariant on a real engine?

c) *Closed loop – offline model of Exercise 2.2.* A two-point controller with integrator changes q with slope $-r$ while the sensor reports rich ($U > 0.45\,\text{V}$) and $+r$ while lean (period 10 ms, 8-bit PWM). Derive the period and the λ amplitude of the resulting oscillation as functions of T_d, r and the lag time constants (hint: a ramp passing a first-order lag τ is delayed by τ). Implement the controller in `closed_loop()`, compare with your prediction for $T_d = 50 \ldots 800\,\text{ms}$, and explain what happens to the mean of q when the unmetered air changes. Extend the prediction to a controller with an additional jump $\pm P$ at every switching of the sensor.

Solution

a) Crankshaft inertia.

Step 1: *Euler.* Replace $\dot{n}$ by $(n[k] - n[k-1])/T$ and evaluate the right-hand side at $k-1$: $n[k] = n[k-1] + \frac{T}{\tau}(Kq[k-1] - n[k-1]) = (1 - \frac{T}{\tau})n[k-1] + \frac{T}{\tau}Kq[k-1]$.

Step 2: *Exact discretisation.* Assume q is constant over one step, $q(t) = q[k-1]$ for $t \in [(k-1)T, kT)$. Then the ODE has the constant input $Kq[k-1]$, and its solution relaxes exponentially from the start value towards $Kq[k-1]$:

$$n(t) = Kq[k-1] + \left(n((k-1)T) - Kq[k-1]\right)\mathrm{e}^{-(t-(k-1)T)/\tau}.$$

Step 3: Evaluate at $t = kT$ and set $a = \mathrm{e}^{-T/\tau}$:

$$n[k] = a\,n[k-1] + (1-a)\,Kq[k-1].$$

With $T = 1\,\text{ms}$, $\tau = 0.4\,\text{s}$: $a = 0.997503$. Euler has $a = 1 - T/\tau = 0.9975$, the first-order Taylor approximation of $\mathrm{e}^{-T/\tau}$.

Step 4: *Simulate and measure τ.* The speed rises from $K \cdot 1 = 2000\,\text{rpm}$ to $K \cdot 1.5 = 3000\,\text{rpm}$. The 63 % point $2000 + 0.632 \cdot 1000 = 2632\,\text{rpm}$ is reached at $t = 0.900\,\text{s}$, i.e. 0.400 s after the step $\Rightarrow \tau = 0.400\,\text{s}$.

Step 5: *Compare at $t = 0.6\,\text{s}$.* Analytic: $2000 + 1000(1 - \mathrm{e}^{-0.1/0.4}) = 2221.20\,\text{rpm}$. Exact discretisation: 2221.20 rpm (identical, as derived). Euler: 2221.44 rpm, off by 0.24 rpm – negligible at $T/\tau = 1/400$.

Step 6: *Static or dynamic?* The speed after the step depends on how long ago the fuel changed, i.e. on past inputs ⇒ dynamic system (with memory), LTI in this model. The difference equation is the exponential smoother of Sheet 3.

Program output:

```
(a) fuel step 1.0 -> 1.5 at t = 0.5 s: n = 2000 -> 2998.1 rpm (t = 3 s)
    63% point reached at t = 0.900 s -> tau = 0.400 s
    n(0.6 s): Euler 2221.44, exact 2221.20, analytic 2221.20 rpm
    difference equation: n[k] = 0.997503 n[k-1] + 0.002497 * K q[k-1]
```

b) Lambda path. `lp_step()` executes the five blocks in signal order, one millisecond per call:

```
80 static double lp_step(lam_path_t *p, double q, double *lam_true)
81 {
82     double lam, lam_d;
83     p->qf += (q - p->qf) / 20.0;                        /* wall film, 20 ms  */
84     lam = (1.0 + p->air) / (p->qf > 0.05 ? p->qf : 0.05);   /* static, NL */
85     if (lam > 2.0) lam = 2.0;
86     if (lam < 0.5) lam = 0.5;
87     p->buf[p->w] = lam;                                  /* transport delay */
88     lam_d = p->buf[(p->w - p->delay + MAXD) % MAXD];
89     p->w = (p->w + 1) % MAXD;
90     p->ls += (lam_d - p->ls) / 30.0;                     /* sensor lag, 30 ms */
91     *lam_true = lam;
92     return es_lambda_voltage(p->ls);                     /* static, NL     */
93 }
```

Step 1: *Wall film* (line 83): first-order lag with $\tau = 20\,\text{ms}$, written as $q_f \mathrel{+}= (q - q_f)/20$ (Euler with $T/\tau = 1/20$) – dynamic, linear, TI.

Step 2: *Mixture* (line 84): $\lambda = (1 + a)/q_f$ – static, *nonlinear* (hyperbola).

Step 3: *Transport delay* (lines 87–89): ring buffer, read the value written T_d steps ago – dynamic (memory), linear, TI.

Step 4: *Sensor lag* (line 90): first-order lag, $\tau = 30\,\text{ms}$ – dynamic, linear, TI.

Step 5: *Switching characteristic* (line 92): static, strongly nonlinear (slope 80 V per unit λ at $\lambda = 1$, saturating at 0.1/0.9 V).

Response to the fuel step $1.00 \to 1.10$ ($a = 0.05$, $T_d = 200\,\text{ms}$), decomposed block by block:

Step 6: λ jumps from $1.05/1.00 = 1.05$ towards $1.05/1.10 = 0.9545$. It crosses 1 when $q_f = 1.05$, i.e. when the wall film has made half of its step: $1 - \mathrm{e}^{-t/20\,\text{ms}} = 0.5 \Rightarrow t = 20\,\text{ms} \cdot \ln 2 = 13.9\,\text{ms}$ (simulation: 13 ms).

Step 7: The transport delay adds 200 ms.

Step 8: The sensor lag must bring λ_s below the 0.45-V point, $0.5 + 0.4 \tanh \frac{1-\lambda_s}{0.005} = 0.45 \Rightarrow \lambda_s = 1 + 0.005\,\text{artanh}(0.125) = 1.0006$: another $\approx 27\,\text{ms}$. Total: 240 ms (simulation).

Step 9: *Double step* $1.00 \to 1.20$: λ crosses 1 when q_f has made only a quarter of its step: $20\,\text{ms} \cdot \ln \frac{4}{3} = 5.8\,\text{ms}$; and since the delayed λ now lies further below 1, the sensor lag also crosses earlier. Total: 223 ms.

```
(b) fuel step 1.00 -> 1.10 at t = 100 ms (air +5 %, Td = 200 ms)
    lambda 1.0500 -> 0.9545, crosses 1 after 13 ms (wall film)
    sensor crosses 0.45 V after 240 ms
    with step 1.00 -> 1.20 (double size): crosses after 223 ms, final U = 0.900 V (step 1.10:
    0.900 V)
```

Proofs of nonlinearity. (i) Doubling the input step does not double the output: the final value is 0.9 V in both cases. (ii) The response *time* depends on the step size (240 vs. 223 ms). An LTI system would only scale its step response. The sensor tells only "rich" or "lean", not *how* rich.

Time variance on a real engine. T_d is the travel time of the gas to the sensor, roughly $\propto$ 1/(mass flow). At changing speed or load T_d changes, so the system is time-variant.

c) Closed loop.

Step 1: *Lump the dynamics.* A ramp passing a first-order lag τ is delayed by τ. For ramps the chain therefore acts like one dead time $T_l = T_d + \tau_f + \tau_s = T_d + 50\,\text{ms}$. Let $q^* = 1 + a$.

Step 2: *Overshoot.* When q ramps up through q^*, the sensor still reports lean for T_l. At the switching $q = q^* + rT_l$.

Step 3: *Return.* The controller ramps down; q is back at q^* after T_l, and the sensor sees this another T_l later. At that switching $q = q^* - rT_l$.

Step 4: *Period and amplitude.* Each half period lasts $2T_l$:

$$T_{\text{period}} = 4\,T_l, \qquad q = q^* \pm rT_l, \qquad \lambda \approx 1 \mp \frac{rT_l}{q^*}.$$

Step 5: *With jump $P < rT_l$.* After the switching q is only $rT_l - P$ away from q^* and reaches it after $T_l - P/r$. The half period shortens by P/r: $T_{\text{period}} = 4T_l - 2P/r$. The peak deviation stays rT_l (it builds up before the switching).

Implementation (`closed_loop()`, controller every 10 steps):

```
if (k % 10 == 0) {                              /* controller, 10 ms */
    int rich = u > 0.45;
    if (rich != rich_old) q += rich ? -pjump : pjump;
    q += (rich ? -rate : rate) * 0.010;
    qpwm = 2.0 * floor(255.0 * q / 2.0 + 0.5) / 255.0;   /* 8-bit PWM */
    rich_old = rich;
}
u = lp_step(&p, qpwm, &lam);
```

Simulation ($r = 0.2/\text{s}$, $+5\,\%$ air, $T_l = T_d + 50\,\text{ms}$):

T_d / ms	period (sim.)	$4T_l$	λ range (sim.)
50	0.38 s	0.40 s	0.985…1.017
100	0.58 s	0.60 s	0.977…1.028
200	0.98 s	1.00 s	0.958…1.046
400	1.78 s	1.80 s	0.924…1.091
800	3.38 s	3.40 s	0.864…1.193

Step 6: *Compare.* The prediction fits within 2–5 %. The simulated periods are 20 ms shorter. This comes from the 8-bit PWM: q changes in stairs of $2/255 = 0.0078$, not as a smooth ramp. With unquantised q the same model gives exactly $4T_l$ (0.60, 1.00, 1.80, 3.40 s). The offset of the sensor threshold ($\lambda_s = 1.0006$) does not matter: it makes one switching earlier and the next one later by the same amount.

Step 7: *Unmetered air* $0/+5/+10\,\%$: mean $q = 0.9989$ / 1.0495 / 1.0986 $\approx 1 + a$, period unchanged (0.98 s). The integrator drives the *mean* mixture to $\lambda = 1$ whatever the disturbance (integral action).

Step 8: *With jump.* $P = 0.03$, $r = 0.2/\text{s}$: period 0.70 s (prediction $1.0 - 0.3 = 0.70\,\text{s}$), λ 0.958…1.045. $P = 0.025$, $r = 0.1/\text{s}$: period 0.58 s, λ 0.978…1.027 (prediction 0.5 s, $\pm 2.4\,\%$). Here $P \approx rT_l$, the jump lands almost on q^*, and the lagged signal approaches q^* only exponentially – the ramp rule of Step 1 no longer applies, so the period is longer than predicted. The jump allows a smaller ramp, i.e. a smaller amplitude at the same or a higher frequency.